

```
!pip install spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.24.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.4)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.2.25)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (0.1.5)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy) (0.24.1)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (7.5.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy) (2.1.2)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (8.3.1)
Requirement already satisfied: shellingham=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (4.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer>=0.24.0->typer-slim<1.0.0,>=0.3.0->spacy) (0.1.2)
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
    12.8/12.8 MB 89.5 MB/s eta 0:00:00
```

✓ Download and installation successful

You can now load the package via `spacy.load('en_core_web_sm')`

⚠ Restart to reload dependencies

If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")  
print("Model loaded successfully!")
```

Model loaded successfully!

```
sentences = [  
    "The cat sits on the mat.",  
    "She is reading a book.",  
    "I went to the market and bought fruits.",  
    "Although it was raining, we went outside.",  
    "He plays cricket.",  
    "Do you like coffee?",  
    "The cake was baked by her.",  
    "They are watching a movie.",  
    "She sings beautifully.",  
    "The teacher explained the lesson clearly.",  
    "When I arrived, they had left.",  
    "He is writing a program.",  
    "The car was repaired by the mechanic.",  
    "We completed the project on time.",  
    "Why are you late?"  
]
```

```
for sent in sentences:  
    doc = nlp(sent)  
    print("\nSentence:", sent)  
  
    for token in doc:  
        print(token.text, token.lemma_, token.pos_, token.dep_, token.head.text)
```

```

is be AUX aux writing
writing write VERB ROOT writing
a a DET det program
program program NOUN dobj writing
. . PUNCT punct writing

```

```

Sentence: The car was repaired by the mechanic.
The the DET det car
car car NOUN nsubjpass repaired
was be AUX auxpass repaired
repaired repair VERB ROOT repaired
by by ADP agent repaired
the the DET det mechanic
mechanic mechanic NOUN pobj by
. . PUNCT punct repaired

```

```

Sentence: We completed the project on time.
We we PRON nsubj completed
completed complete VERB ROOT completed
the the DET det project
project project NOUN dobj completed
on on ADP prep completed
time time NOUN pobj on
. . PUNCT punct completed

```

```

Sentence: Why are you late?
Why why SCONJ advmod are
are be AUX ROOT are
you you PRON nsubj are
late late ADJ acomp are
? ? PUNCT punct are

```

```

import pandas as pd

data = []

for sent in sentences:
    doc = nlp(sent)
    for token in doc:
        data.append([
            sent,
            token.text,
            token.lemma_,
            token.pos_,
            token.dep_,
            token.head.text
        ])

df = pd.DataFrame(data, columns=["Sentence", "Token", "Lemma", "POS", "Dependency", "Head"])
df.head(20)

```

	Sentence	Token	Lemma	POS	Dependency	Head
0	The cat sits on the mat.	The	the	DET	det	cat
1	The cat sits on the mat.	cat	cat	NOUN	nsubj	sits
2	The cat sits on the mat.	sits	sit	VERB	ROOT	sits
3	The cat sits on the mat.	on	on	ADP	prep	sits
4	The cat sits on the mat.	the	the	DET	det	mat
5	The cat sits on the mat.	mat	mat	NOUN	pobj	on
6	The cat sits on the mat.	.	.	PUNCT	punct	sits
7	She is reading a book.	She	she	PRON	nsubj	reading
8	She is reading a book.	is	be	AUX	aux	reading
9	She is reading a book.	reading	read	VERB	ROOT	reading
10	She is reading a book.	a	a	DET	det	book
11	She is reading a book.	book	book	NOUN	dobj	reading
12	She is reading a book.	.	.	PUNCT	punct	reading
13	I went to the market and bought fruits.	I	I	PRON	nsubj	went
14	I went to the market and bought fruits.	went	go	VERB	ROOT	went
15	I went to the market and bought fruits.	to	to	ADP	prep	went
16	I went to the market and bought fruits.	the	the	DET	det	market
17	I went to the market and bought fruits.	market	market	NOUN	pobj	to
18	I went to the market and bought fruits.	and	and	CCONJ	cc	went
19	I went to the market and bought fruits.	bought	buy	VERB	conj	went

Sentence Structure Identification

In this step, the structure of sentences is analyzed using dependency relations obtained from spaCy.

Example 1: Sentence: "The cat sits on the mat." Subject: cat (nsubj) Main Verb: sits (ROOT) Object: mat (pobj) Modifiers: The (det), on (prep) Explanation: The subject "cat" performs the action "sits". The phrase "on the mat" acts as a prepositional modifier providing additional information.

Example 2: Sentence: "She is reading a book." Subject: She (nsubj) Main Verb: reading (ROOT) Object: book (dobj) Modifiers: is (aux), a (det) Explanation: The auxiliary verb "is" supports the main verb "reading". The object "book" receives the action.

Example 3: Sentence: "The teacher explained the lesson clearly." Subject: teacher (nsubj) Main Verb: explained (ROOT) Object: lesson (dobj) Modifiers: The (det), clearly (advmod) Explanation: The adverb "clearly" modifies the verb "explained" and provides more information about how the action is performed.

Example 4: Sentence: "Although it was raining, we went outside." Subject: we (nsubj) Main Verb: went (ROOT) Object: outside (advmod) Modifiers: Although (mark), raining (advcl) Explanation: This is a complex sentence. The clause "Although it was raining" adds extra information and creates a more complex dependency structure.

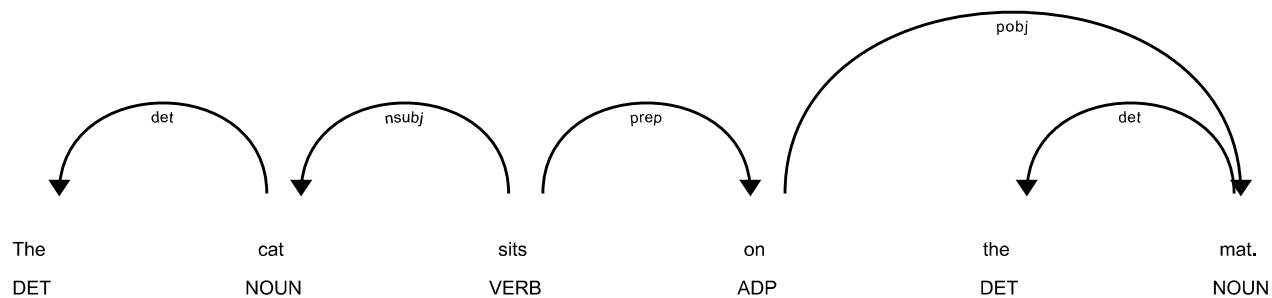
Example 5: Sentence: "The cake was baked by her." Subject: cake (nsubjpass) Main Verb: baked (ROOT) Object: her (pobj) Modifiers: was (auxpass), by (prep) Explanation: This is a passive sentence. The subject "cake" receives the action, while "her" is the actual doer of the action.

How Dependency Relations Help: Dependency labels such as nsubj, dobj, ROOT, amod, and prep clearly show the grammatical roles of words in a sentence. They help in identifying the subject, verb, object, and modifiers easily.

Difference Between Simple and Complex Sentences: Simple sentences contain a single clause with a straightforward structure. Complex sentences contain multiple clauses connected using conjunctions, resulting in deeper and more complicated dependency trees.

```
from spacy import displacy

doc = nlp("The cat sits on the mat.")
displacy.render(doc, style="dep", jupyter=True)
```



Comparative Analysis of Sentence Structures

In this experiment, different types of sentences were analyzed using spaCy dependency parsing. The results show clear differences between simple, compound, complex, interrogative, and passive sentences.

Simple sentences such as "He plays cricket" produced very simple dependency trees with a single subject (nsubj) and a root verb (ROOT). These sentences are easy to analyze because they contain only one clause.

Compound sentences like "I went to the market and bought fruits" showed more than one verb connected using conjunctions. This resulted in slightly more complex tree structures with multiple actions linked together.

Complex sentences such as "Although it was raining, we went outside" produced deeper dependency trees because they contain dependent clauses. Words like "Although" introduce additional relationships, making the structure more complex.

Interrogative sentences like "Do you like coffee?" showed different structures where auxiliary verbs (aux) play an important role. The word order is also different compared to simple sentences.

Passive voice sentences such as "The cake was baked by her" showed significant changes in dependency relations. The object in active voice becomes the subject in passive voice, and the actual doer is introduced using a prepositional phrase (prep).

Modifiers such as adjectives (amod) and adverbs (advmod) added extra branches to the dependency tree. For example, in "She sings beautifully", the word "beautifully" modifies the verb and adds more detail.

Overall, simple sentences produced the simplest trees, while complex and passive sentences produced more detailed and deeper structures. Dependency parsing helps clearly understand how words are related in different sentence types.

LAB REPORT: Syntactic Analysis using spaCy

1. Objective: The objective of this lab is to understand syntactic analysis in Natural Language Processing (NLP) using spaCy. The aim is to identify grammatical relationships between words using dependency parsing and visualize sentence structures.
2. Dataset Description: The dataset consists of 15 sentences of different types including simple, compound, complex, interrogative, and passive sentences. These sentences were collected from daily conversations, general English examples, and basic news-style statements. The dataset was designed to include a variety of grammatical structures for better analysis.
3. Installation and Setup: The following tools were used:
 - Python 3.x
 - spaCy library
 - en_core_web_sm language model
 - Google Colab

Commands used: `pip install spacy` `python -m spacy download en_core_web_sm`

4. Dependency Parsing Results: Each sentence was processed using spaCy to extract tokens, lemmas, part-of-speech tags, dependency labels, and head words. The results were organized in a tabular format for better understanding.
5. Dependency Tree Visualization: Dependency trees were generated using spaCy's displaCy visualizer. These visualizations helped in understanding the grammatical structure of sentences and how different words are connected.
6. Sentence Structure Analysis: Using dependency labels such as nsubj (subject), ROOT (main verb), dobj (object), and amod (modifier), the structure of each sentence was analyzed. This helped identify key components like subject, verb, object, and modifiers.
7. Comparative Analysis: Simple sentences had straightforward structures, while compound and complex sentences had multiple clauses and deeper trees. Passive sentences showed changes in subject-object relationships. Modifiers increased the complexity of the dependency tree.
8. Conclusion: This lab provided a clear understanding of syntactic analysis using spaCy. Dependency parsing is useful in understanding sentence structure, which is important in many NLP applications such as machine translation, chatbots, and text analysis.

Start coding or [generate](#) with AI.

